

```

rm(list = ls())

# Local github
codepath <- c("/media/larryleon/My Projects/GitHub/Forest-Search/R/")
source(paste0(codepath, "source_forestsearch_v0.R"))
source_fs_functions(file_loc = codepath)

library(kableExtra)
library(knitr)
library(ggplot2)
library(gridExtra)
library(cubature)
library(aVirtualTwins)
library(randomForest)
library(survival)
library(survminer)
library(grf)
library(policytree)
library(data.table)
library(plyr)
library(dplyr)
library(glmnet)
library(corrplot)

maxFollow <- 84
cens.type <- "weibull"

# m1 -censoring adjustment
muC.adj <- log(1.5)
k.z3 <- 1
k.treat <- 0.9

z1_frac <- 0.25 # Default model index 'm1' (The 1st quartile of z1=er)
pH_super <- 0.125 # non-NULL re-defines z1_frac

if (is.null(pH_super)) {
  # pH_check<-with(gbbsg,mean(pgr<=quantile(pgr,c(z3_frac),1,0) &
  # er<=quantile(er,z1_frac)))
  pH_check <- with(gbbsg, mean(meno == 0 & er <= quantile(er, z1_frac)))
  cat("Underlying pH_super", c(pH_check), "\n")
}
# pH_super specified If pH_super then override z1_frac and find z1_frac to
# yield pH_super

if (!is.null(pH_super)) {
  # Approximate Z1 quantile to yield pH proportion
  z1_q <- uniroot(propH.obj4, c(0, 1), tol = 1e-04, pH.target = pH_super)$root
  # pH_check<-with(gbbsg,mean(pgr<=quantile(pgr,c(z3_frac),1,0) &
  # er<=quantile(er,z1_q)))
  pH_check <- with(gbbsg, mean(meno == 0 & er <= quantile(er, z1_q)))
  cat("pH", c(pH_check), "\n")
  rel_error <- (pH_super - pH_check)/pH_super
  if (abs(rel_error) >= 0.1)
    stop("pH_super approximation relative error exceeds 10%")
  z1_frac <- z1_q
}

```

```

    cat("Underlying pH_super", c(pH_check), "\n")
  }

## pH 0.122449
## Underlying pH_super 0.122449

# Bootstrap on log(hr) scale converted to HR (est.loghr=TRUE & est.scale='hr')
t.start.all <- proc.time()[3]

##### Forest search criteria
hr.threshold <- 1.25 # Initial candidates
hr.consistency <- 1 # Candidates for many splits
pconsistency.threshold <- 0.9
stop.threshold <- 0.95
maxk <- 2
nmin.fs <- 60
pstop_futile <- 0.5
# Limit timing for forestsearch
max.minutes <- 3
m1.threshold <- Inf # Turning this off (Default)
# pconsistency.threshold<-0.70 # Minimum threshold (will choose max among
# subgroups satisfying)
fs.splits <- 400 # How many times to split for consistency
# vi is % factor is selected in cross-validation --> higher more important
vi.grf.min <- 0.2
# Null, turns off grf screening
d.min <- 10 # Min number of events for both arms (d0.min=d1.min=d.min)
# default=5 Virtual twins analysis Counter-factual difference (C-E) >=
# vt.threshold Large values in favor of C (control)
vt.threshold <- 0.225 # For VT delta
treat.threshold <- 0

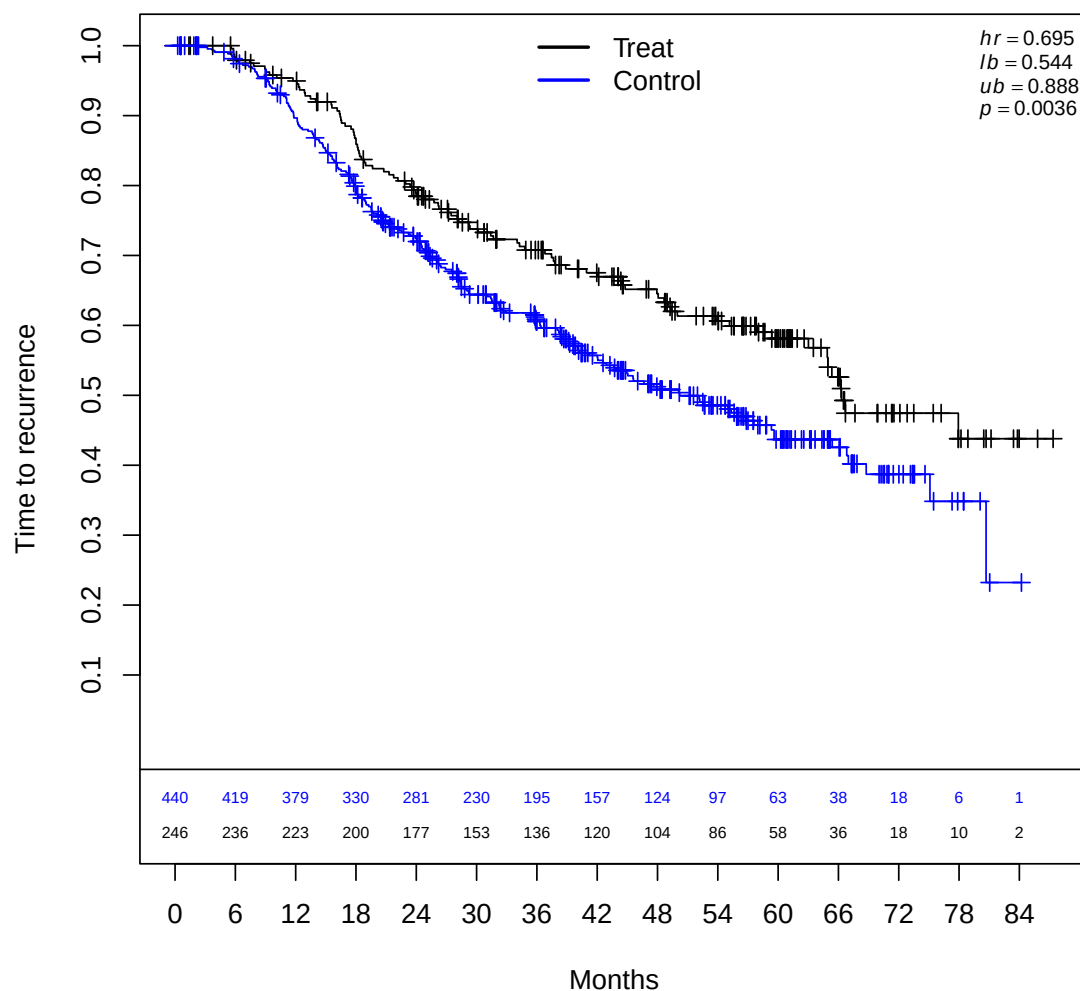
maxdepth <- 2
n.min <- 60
ntree <- 1000
# GRF criteria
dmin.grf <- 12 # For GRF delta
# Note: For CRT this represents dmin.grf/2 RMS for control (-dmin.grf/2 for
# treatment)
frac.tau <- 0.6

outcome.name <- c("y.sim")
event.name <- c("event.sim")
id.name <- c("id")
treat.name <- c("treat")

cox.formula.sim <- as.formula(paste("Surv(y.sim,event.sim)~treat"))
cox.formula.adj.sim <- as.formula(paste("Surv(y.sim,event.sim)~treat+v1+v2+v3+v4+v5"))

mod.harm <- "alt"
hrH.target <- 2.5
# out.loc = NULL turns off file creation
N <- 700
this.dgm <- get.dgm4(mod.harm = mod.harm, N = N, k.treat = k.treat, hrH.target = hrH.target,
  cens.type = cens.type, out.loc = NULL, details = TRUE, parms_torand = FALSE)

```



```
## Super-population empirical harm and non-harm hazard ratios= 2.499996 0.6466405
## Causal HR (empirical ITT)= 0.7125876
```

```
file_out <- c("output/sim-FS4_Lasso_ex11.Rdata")

n_add_noise <- 3

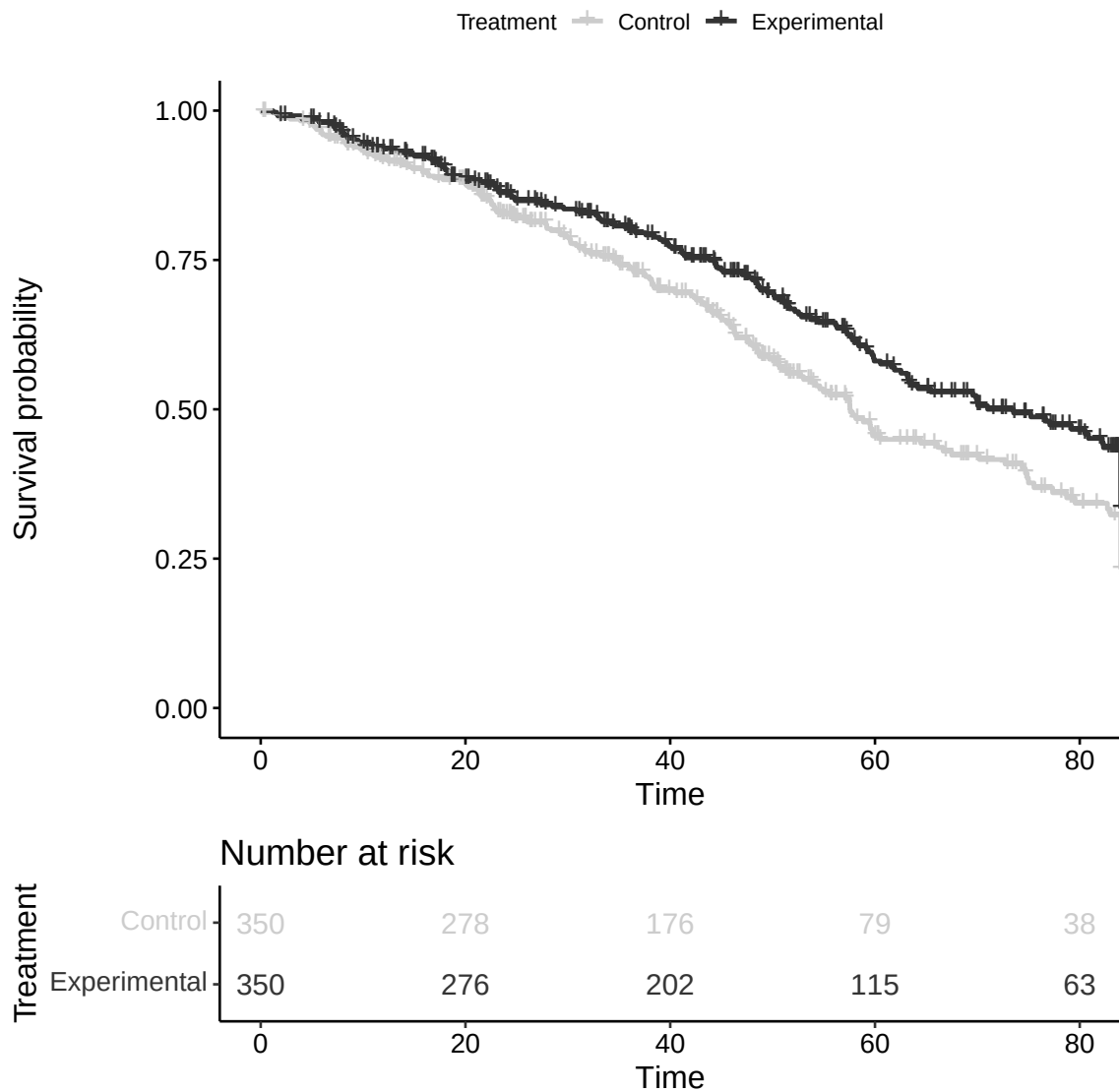
sim <- 23

dgm <- this.dgm$dgm

x <- sim_aftm4_gbsg(dgm = dgm, n = N, maxFollow = maxFollow, muC.adj = muC.adj, simid = sim)

kmfit <- survfit(Surv(y.sim, event) ~ treat, data = x)

ggsurvplot(kmfit, data = x, main = "K-M curves for simulated data", legend = "top",
  legend.title = "Treatment", legend.labs = c("Control", "Experimental"), palette = "grey",
  risk.table = TRUE, risk.table.col = "strata")
```



```

if (n_add_noise == 0) {
  confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade3")
}

if (n_add_noise == 5) {
  set.seed(8316951 + 1000 * sim)
  # Add 5 noise
  x$noise1 <- rnorm(N)
  x$noise2 <- rnorm(N)
  x$noise3 <- rnorm(N)
  x$noise4 <- rnorm(N)
  x$noise5 <- rnorm(N)
  confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade3", "noise1",
    "noise2", "noise3", "noise4", "noise5")
}

if (n_add_noise == 3) {
  set.seed(8316951 + 1000 * sim)

```

```

# Add 3 noise
x$noise1 <- rnorm(N, sd = 1)
x$noise2 <- rnorm(N, sd = 1)
x$noise3 <- rnorm(N, sd = 1)
confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade3", "noise1",
                      "noise2", "noise3")
}

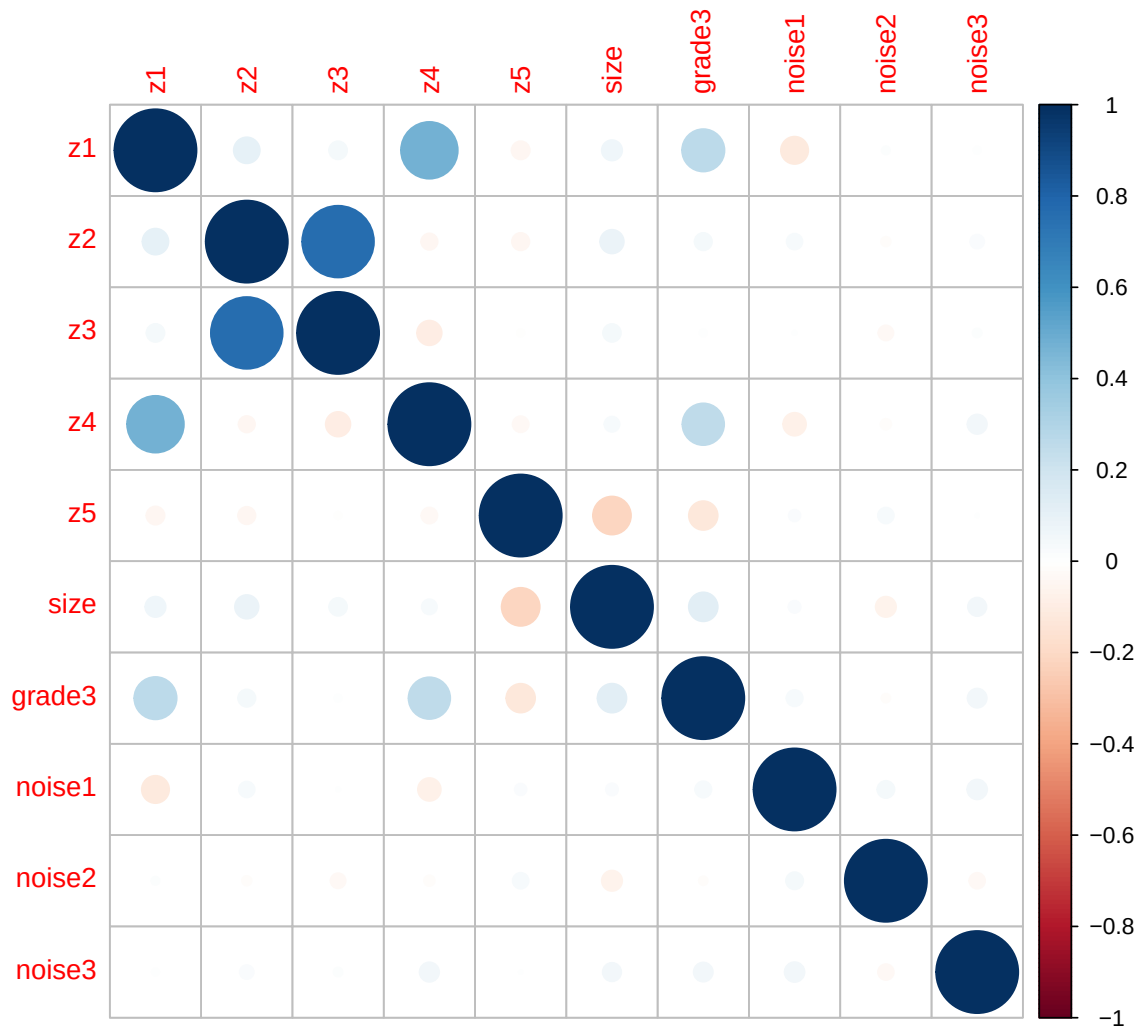
# More challenging which_replace <- which(confounders.name=='z1')
# confounders.name[which_replace] <- 'er'

cox.formula.check <- as.formula(paste("Surv(y.sim,event.sim)~treat+v1+v2+v3+v4+v5+size+grade3+noise1",
                                     "noise2", "noise3"))
coxph(cox.formula.check, data = x)

## Call:
## coxph(formula = cox.formula.check, data = x)
##
##              coef exp(coef)    se(coef)      z        p
## treat  -4.288e-01  6.513e-01  1.088e-01 -3.941  8.12e-05
## v11      7.018e-02  1.073e+00  1.274e-01  0.551  0.5817
## v21     -3.014e-01  7.398e-01  1.680e-01 -1.795  0.0727
## v31      3.932e-01  1.482e+00  1.689e-01  2.328  0.0199
## v41      8.918e-01  2.439e+00  1.206e-01  7.392 1.44e-13
## v51     -1.047e+00  3.509e-01  1.126e-01 -9.303 < 2e-16
## size    -4.146e-06  1.000e+00  3.407e-03 -0.001  0.9990
## grade3  -1.816e-01  8.339e-01  1.252e-01 -1.450  0.1470
## noise1   2.406e-02  1.024e+00  5.405e-02  0.445  0.6562
## noise2  -4.244e-02  9.585e-01  5.610e-02 -0.756  0.4494
## noise3   5.278e-02  1.054e+00  5.763e-02  0.916  0.3598
##
## Likelihood ratio test=171.6 on 11 df, p=< 2.2e-16
## n= 700, number of events= 372

Zm <- cor(as.matrix(x[, c(confounders.name)]))
corrplot(Zm)

```



```
# Options Allconfounders.name is list of confounders within analysis dataset
# (1) use_lasso=TRUE & use_grf=FALSE Lasso used to possibly reduce dimension
# Any continuous factors are cut at medians (2) use_lasso=TRUE & use_grf=TRUE
# Lasso used to reduce dimension Continuous covariates are cut at medians
# However, if GRF selects a covariate cut then only that cut is used: For
# example if 'age <= median(ag)' is called for per Lasso but GRF includes 'age
# <= 54', then only the latter is used (3) use_grf_only = TRUE (overrides
# use_lasso and use_grf) Only factors selected via GRF are used (4) use_lasso =
# F & use_grf =T Median cuts (unless selected via GRF) as in (2) However no
# possible dimension reduction via lasso All categorical factors included

use_lasso <- TRUE
use_grf <- TRUE
use_grf_only <- FALSE

fs.est <- forestsearch(df.analysis = x, Allconfounders.name = confounders.name, details = TRUE,
  use_lasso = use_lasso, use_grf = use_grf, use_grf_only = use_grf_only, dmin.grf = 10,
  frac.tau = 0.6, conf_force = NULL, outcome.name = outcome.name, treat.name = treat.name,
```

```

event.name = event.name, id.name = id.name, n.min = nmin.fs, hr.threshold = hr.threshold,
hr.consistency = hr.consistency, fs.splits = fs.splits, d0.min = d.min, d1.min = d.min,
pstop_futile = pstop_futile, pconsistency.threshold = pconsistency.threshold,
stop.threshold = stop.threshold, max.minutes = max.minutes, maxk = maxk, by.risk = 12,
plot.sg = TRUE, vi.grf.min = vi.grf.min)

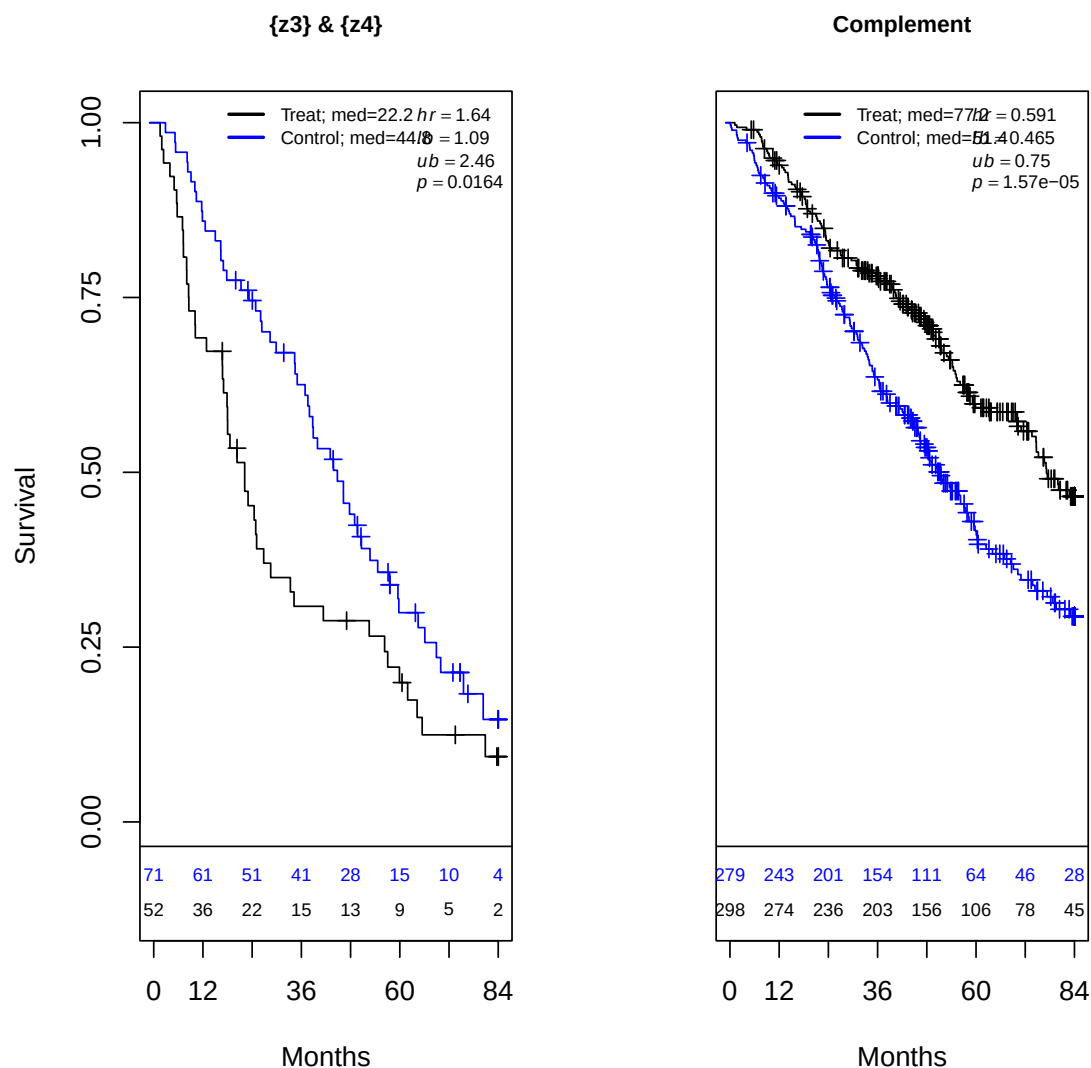
## tau, maxdepth= 49.69171 2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 1         2   -5.156029   413.000000   1.419432    5.156029   413.000000
## 2         3    1.882604   287.000000   1.735410   -1.882604   287.000000
## 3         4   -5.630643   204.000000   2.071336    5.630643   204.000000
## 4         5    5.629364   152.000000   2.514496   -5.629364   152.000000
## 5         6   -4.649491   332.000000   1.494023    4.649491   332.000000
##   treated.se      diff Nsg depth
## 1   1.419432 -10.312058 413     1
## 2   1.735410   3.765209 287     1
## 3   2.071336 -11.261285 204     2
## 4   2.514496  11.258728 152     2
## 5   1.494023  -9.298982 332     2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 4         5    5.629364   152.000000   2.514496   -5.629364   152.000000
##   treated.se      diff Nsg depth
## 4   2.514496  11.25873 152     2
## [1] "noise2 <= -0.08" "z3 <= 0"          "noise3 <= 1.7"
## [1] "z3 <= 0"
## # of continuous/categorical characteristics 4 6
## Continuous characteristics: size noise1 noise2 noise3
## Categorical characteristics: z1 z2 z3 z4 z5 grade3
## 10 x 1 sparse Matrix of class "dgCMatrix"
##           s0
## z1         .
## z2         .
## z3      0.112025756
## z4      0.761514747
## z5     -0.863449482
## size       .
## grade3     .
## noise1 0.001887115
## noise2  .
## noise3  .
## Cox-LASSO selected: z3 z4 z5 noise1
## Cox-LASSO not selected: z1 z2 size grade3 noise2 noise3
## Median cuts (after Lasso if use_lasso) noise1
## Categorical (after Lasso if use_lasso) z3 z4 z5
## Prior to removing (if in GRF): noise1
## Left (to cut at median) after removing others if in GRF: noise1
## Initial GRF cuts included noise2 <= -0.08 z3 <= 0 noise3 <= 1.7
## # of candidate subgroup factors= 6
## [1] "noise1 <= median(noise1)" "noise2 <= -0.08"
## [3] "noise3 <= 1.7"           "z3"
## [5] "z4"                     "z5"
## Confounders per grf screening q4 q2 q6 q1 q5
##   FSconfounders.name      vi.cs      vi_ratio
## 4                q4 0.3456293701 2.073776221
## 2                q2 0.1907045970 1.144227582
## 6                q6 0.1713741995 1.028245197

```

```

## 1          q1 0.1558734452 0.935240671
## 5          q5 0.1360668172 0.816400903
## 3          q3 0.0003515711 0.002109426
## Number of unique levels (L) and possible subgroups= 10 1023
## # of subgroups based on # variables > k.max and excluded (per million) 0.000968
## k.max= 2
## Events criteria for control,exp= 10 10
## # of subgroups with events less than criteria: control, experimental 5 5
## # of subgroups with sample size less than criteria 5
## # of subgroups meeting all criteria = 50
## # of subgroups fitted (Cox model estimable) = 50
## *Subgroup Searching Minutes=* 0.00215
## Number of subgroups meeting HR threshold 1
## # of candidate subgroups (meeting HR criteria) = 1
## Subgroups (1st 10) meeting overall screening thresholds (HR, m1) sorted by HRs= Inf
##      K   n   E d1    m1    m0   HR L(HR) U(HR) q4.0 q4.1 q2.0 q2.1 q6.0 q6.1
## 1:  2 123 96 44 22.18 44.78 1.64  1.09  2.46    0    1    0    0    0    0
## 2: NA  NA NA NA    NA    NA   NA    NA    NA   NA   NA   NA   NA   NA   NA
## 3: NA  NA NA NA    NA    NA   NA    NA    NA   NA   NA   NA   NA   NA   NA
## 4: NA  NA NA NA    NA    NA   NA    NA    NA   NA   NA   NA   NA   NA   NA
## 5: NA  NA NA NA    NA    NA   NA    NA    NA   NA   NA   NA   NA   NA   NA
## 6: NA  NA NA NA    NA    NA   NA    NA    NA   NA   NA   NA   NA   NA   NA
## 7: NA  NA NA NA    NA    NA   NA    NA    NA   NA   NA   NA   NA   NA   NA
## 8: NA  NA NA NA    NA    NA   NA    NA    NA   NA   NA   NA   NA   NA   NA
## 9: NA  NA NA NA    NA    NA   NA    NA    NA   NA   NA   NA   NA   NA   NA
## 10: NA NA NA NA    NA    NA   NA    NA    NA   NA   NA   NA   NA   NA   NA
##      q1.0 q1.1 q5.0 q5.1
## 1:    0    0    0    1
## 2:   NA   NA   NA   NA
## 3:   NA   NA   NA   NA
## 4:   NA   NA   NA   NA
## 5:   NA   NA   NA   NA
## 6:   NA   NA   NA   NA
## 7:   NA   NA   NA   NA
## 8:   NA   NA   NA   NA
## 9:   NA   NA   NA   NA
## 10:  NA   NA   NA   NA
## Consistency 0.9775
## # of splits= 400
## Model, % Consistency Met= {z3} {z4} 0.9775
## Number of subgroups meeting consistency criteria=
##      p.consistency Nsg group.id m.index K   M.1   M.2
## 1:      0.9775 123          1      1 2 {z3} {z4}

```



```
## [1] "{z3}" "{z4}"
## % consistency criteria met= 0.9775
## SG focus= hr
## Subgroup Consistency Minutes= 0.0264
## Subgroup found
## Minutes overall= 0.04795

grf.est <- grf.subg.harm.survival(data = x, confounders.name = confounders.name,
  outcome.name = outcome.name, event.name = event.name, id.name = id.name, treat.name = treat.name,
  n.min = n.min, dmin.grf = dmin.grf, frac.tau = 1, details = TRUE)

## tau, maxdepth= 82.81952 2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 1         2    -9.865316   537.000000   2.271331     9.865316   537.000000
## 2         3     1.431633   163.000000   4.189690    -1.431633   163.000000
## 11        4    -9.421142   520.000000   2.287215     9.421142   520.000000
## 3         6   -16.120361    96.000000   5.206166    16.120361    96.000000
## 4         7    15.311287    79.000000   6.179571   -15.311287    79.000000
##   treated.se      diff Nsg depth
```

```
## 1    2.271331 -19.730633 537    1
## 2    4.189690   2.863266 163    1
## 11   2.287215 -18.842283 520    2
## 3    5.206166 -32.240722  96    2
## 4    6.179571  30.622574  79    2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 4         7    15.311287    79.000000   6.179571  -15.311287    79.000000
##   treated.se      diff Nsg depth
## 4   6.179571 30.62257  79    2
## [1] "z1 <= 0"      "size <= 80" "z3 <= 0"
## [1] "z3 <= 0"
```

```
library(doParallel)
registerDoParallel(parallel::detectCores(logical = FALSE))

cox.formula.boot <- cox.formula.sim
max.minutes <- 7
# Suggest running 50, first ... to get timing estimate

NB <- 1000

df_boot_analysis <- fs.est$df.est

fitH <- get_Cox_sg(df_sg = subset(df_boot_analysis, treat.recommend == 0), cox.formula = cox.formula)
H_obs <- fitH$est_obs # log(hr) scale
seH_obs <- fitH$se_obs
# Hc observed estimates
fitHc <- get_Cox_sg(df_sg = subset(df_boot_analysis, treat.recommend == 1), cox.formula = cox.formula)
Hc_obs <- fitHc$est_obs
seHc_obs <- fitHc$se_obs
rm("fitH", "fitHc")

Ystar_mat <- bootYstar({
  ystar <- get_Ystar(boot)
}, boots = NB, seed = 8316951, counter = "boot", export = fun_arg_list_boot)
# Check dimension
if (dim(Ystar_mat)[1] != NB | dim(Ystar_mat)[2] != nrow(df_boot_analysis)) stop("Dimension of Ystar_")

tB.start <- proc.time()[3]
# Bootstraps
resB <- bootPar({
  ans <- fsboot_forparallel(boot)
}, boots = NB, seed = 8316951, counter = "boot", export = fun_arg_list_boot)
tB.now <- proc.time()[3]
tB.min <- (tB.now - tB.start)/60

doParallel::stopImplicitCluster()

cat("Minutes for Boots", c(NB, tB.min), "\n")
## Minutes for Boots 1000 3.588317

cat("Projection per 1000", c(tB.min * (1000/NB)), "\n")
## Projection per 1000 3.588317

cat("Propn bootstrap subgroups found =", c(sum(!is.na(resB$H_biasadj_1))/NB), "\n")
```

```

## Propn bootstrap subgroups found = 0.945

# How many timmed out
cat("Number timmed out=", c(sum(is.na(resB$H_biasadj_1) & resB$tmins_search > max.minutes)),
    "\n")

## Number timmed out= 0

H_estimates <- get_dfRes(Hobs = H_obs, seHobs = seH_obs, H1_adj = resB$H_biasadj_2,
    ystar = Ystar_mat, cov_method = "standard", cov_trim = 0)
Hc_estimates <- get_dfRes(Hobs = Hc_obs, seHobs = seHc_obs, H1_adj = resB$Hc_biasadj_2,
    ystar = Ystar_mat, cov_method = "standard", cov_trim = 0)

print(H_estimates)

##           H0          sdH0 H0_lower H0_upper          H1          sdH1 H1_lower H1_upper
## 1: 1.639708 0.3378324 1.094943 2.455508 1.408539 0.380832 0.8291373 2.392828

print(Hc_estimates)

##           H0          sdH0 H0_lower H0_upper          H1          sdH1 H1_lower
## 1: 0.5909388 0.0719693 0.4654535 0.7502546 0.6004953 0.1132946 0.4148724
##           H1_upper
## 1: 0.8691698

# H_estimates<-get_dfRes(Hobs=H_obs,seHobs=seH_obs,H1_adj=resB$H_biasadj_1,ystar=Ystar_mat,cov_metho
# Hc_estimates<-get_dfRes(Hobs=Hc_obs,seHobs=seHc_obs,H1_adj=resB$Hc_biasadj_1,ystar=Ystar_mat,cov_m
# print(H_estimates) print(Hc_estimates)

bootit <- list(H_estimates = H_estimates, Hc_estimates = Hc_estimates)
ans3 <- fsBoot.parallel.out(df = df_boot_analysis, FSboots = bootit, cox.formula.boot = cox.formula.
    sim_number = sim)
tall.min <- (tB.now - t.start.all)/60
cat("Overall minutes for analysis", c(tall.min), "\n")

## Overall minutes for analysis 3.716533

save(dgm, x, fs.est, grf.est, bootit, ans3, tall.min, resB, cox.formula.boot, file = file_out)

# ITT causal hazard ratio mean(fs.est$df.est$hlhlin.ratio) H Oracle dfH <-
# subset(fs.est$df.est, flag.harm==1) mean(dfH$hlhlin.ratio) dfH_est <-
# subset(fs.est$df.est, treat.recommend==0) mean(dfH_est$hlhlin.ratio) dfHc <-
# subset(fs.est$df.est, flag.harm==0) mean(dfHc$hlhlin.ratio) dfHc_est <-
# subset(fs.est$df.est, treat.recommend==1) mean(dfHc_est$hlhlin.ratio)

# dfSens <- subset(fs.est$df.est, flag.harm==1 & treat.recommend==0) sensH <-
# nrow(dfSens)/nrow(dfH)

## Confounders evaluated in Forestsearch:
## [1] "noise1 <= median(noise1)" "noise2 <= -0.08"
## [3] "noise3 <= 1.7" "z3"
## [5] "z4" "z5"

```

Table 1 Simulated data example: Cox hazard ratio (hr) estimates for the ITT population and subgroups H and H^c . Cox model estimates are based on subgroups: H true (knowing the actual subgroup, a-priori); the estimated subgroup $\hat{H}$; and the bootstrap (1,000 resamples) bias-correction to $\hat{H}$ estimates, denoted $\hat{H}_{bc}$. Estimates for the complement H^c are defined analogously. The number of subjects in each population (# Subjects) and the number of subjects correctly classified (# Correct) in the true subgroups H and H^c , respectively, are listed.

	HR Estimate	Lower	Upper	# Subjects	# Correct
ITT	0.691	0.563	0.849	700	.
H subgroup estimates					
H_{oracle}	3.456	2.049	5.83	79	.
$\hat{H}$	1.64	1.095	2.456	123	64
$\hat{H}_{bc}$	1.409	0.829	2.394	123	64
H-complement subgroup estimates					
H_{oracle}^c	0.601	0.48	0.753	621	.
$\hat{H}^c$	0.591	0.465	0.75	577	562
$\hat{H}_{bc}^c$	0.6	0.415	0.868	577	562